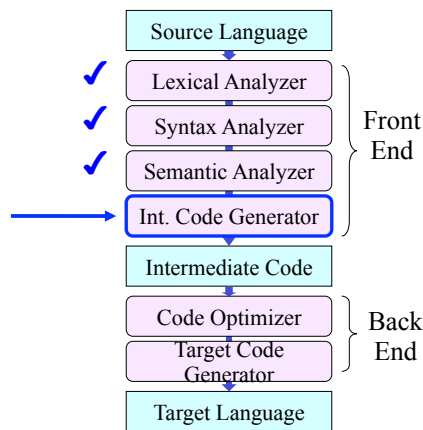
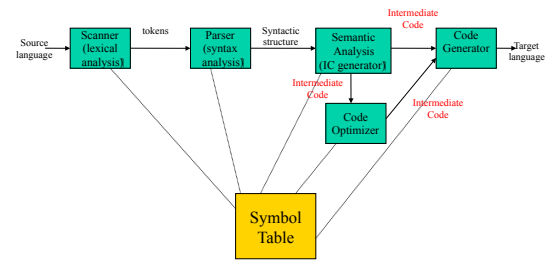


Intermediate Code

Compiler Architecture



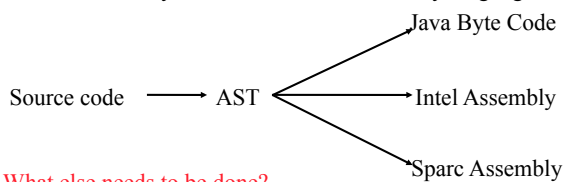
Intermediate Code

- Similar terms: *Intermediate representation*, *intermediate language*
- Ties the front and back ends together

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



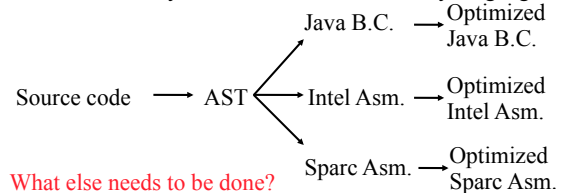
What else needs to be done?

4

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



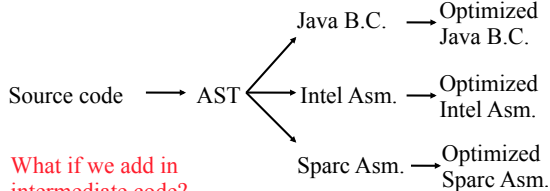
What else needs to be done?

5

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



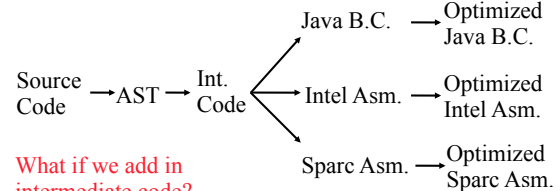
What if we add in intermediate code?

6

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



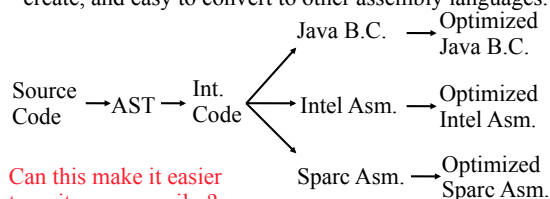
What if we add in intermediate code?

7

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



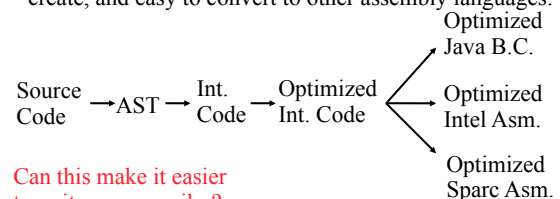
Can this make it easier to write our compiler?

8

What is Intermediate Code?

•Intermediate code is a layer between the front-end and back-end of the compiler.

•It is a simplified assembly language that is easy to create, and easy to convert to other assembly languages.



Can this make it easier to write our compiler?

9

Intermediate Code

- Similar terms: *Intermediate representation*, *intermediate language*
- Is Language and Machine neutral
- Has many forms
- Level depends on how being processed
- More than one intermediate language may be used by a compiler
- **Goal:** Get program closer to machine code without losing information needed to do useful optimizations.

10

High-level vs Low-level

- High-level
 - Expresses high-level structure of program, and is closer to source language
 - Easier to generate from source program
 - Code optimization not straight-forward due to lack of fine-grained representation of source
 - Representation techniques in intermediate languages are a continuum between high and low level.
- Low-level
 - Expresses low-level structure of program, and is closer to target machine
 - Easier to generate target (final) code
 - Harder to generate from source language

11

Intermediate language levels

- | | | |
|--------------------------|--------------------------------|--------------------------|
| • High | • Medium | • Low |
| $t1 \leftarrow a[i,j+2]$ | $t1 \leftarrow j + 2$ | $r1 \leftarrow [fp-4]$ |
| | $t2 \leftarrow i * 20$ | $r2 \leftarrow r1 + 2$ |
| | $t3 \leftarrow t1 + t2$ | $r3 \leftarrow [fp-8]$ |
| | $t4 \leftarrow 4 * t3$ | $r4 \leftarrow r3 * 20$ |
| | $t5 \leftarrow \text{addr } a$ | $r5 \leftarrow r4 + r2$ |
| | $t6 \leftarrow t5 + t4$ | $r6 \leftarrow 4 * r5$ |
| | $t7 \leftarrow *t6$ | $r7 \leftarrow fp - 216$ |
| | | $f1 \leftarrow [r7+r6]$ |

12

What makes a good I.C.?

- A good intermediate code should be easy to translate an AST into, and easy to then convert it to just about any assembly language.
- What does this entail?
 - A narrow interface that contains as few commands as possible, yet is also complete.
 - Designed to be easy to perform common optimizations.
 - Flexible commands that make it easy to identify what aspects of the commands are being used, so they can be retargetted to just about any assembly language.
 - IC operations should not be too closely coupled to any particular machine or architecture for ease in portability.

13

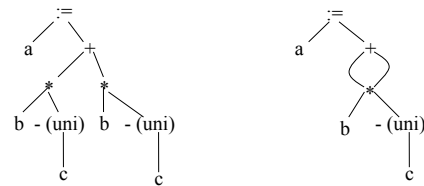
Graphical IRs

- Abstract Syntax Trees (AST) – retain essential structure of the parse tree, eliminating unneeded nodes. Nodes represent operators, children of node represent operands.
- Directed Acyclic Graphs (DAG) – compacted AST to avoid duplication (common subexpressions represented by a single node) – smaller footprint as well
- Control flow graphs (CFG) – explicitly model control flow

14

ASTs and DAGs:

$a := b * -c + b * -c$



15

Linearized IC

- These include stack-based and *three-address-code*.
- The most commonly used intermediate code is *three-address code*.
- It is easiest to use for optimization.
- It gets this name because up to three operands can be used per line.
- **Stack based (one address) – compact**

```

push 2
push y
multiply
push x
subtract
            
```
- **Three address (quadruples) – up to three operands, one operator**

```

t1 <- 2
t2 <- y
t3 <- t1 * t2
t4 <- x
t5 <- t4 - t3
            
```

16

Three-address Code

Format for a typical command is:
 $a = b \text{ OP } c$

Sometimes this is also called quadruples (OP, a, b, c).

Example: $a = (b + c) * (-e)$

```

t1 = b + c      (ADD, t1, b, c)
t2 = -e         (UMINUS, t2, e)
t3 = t1 * t2    (TIMES, t3, t1, t2)
a = t3          (ASSIGN, a, t3)
            
```

Note: t1, t2, and t3 are computer-generated variables.

17

Example: Three-address Code

Binary: $a = b \text{ OP } c$
 Math: ADD, SUB, MUL, DIV, MOD
 Logic: AND, OR
 Relation: EQ, NEQ, LT, GT, LEQ, GEQ

Unary: $a = \text{OP } b$
 Math: MINUS, ASSIGN
 Logic: NOT
 Memory: ALLOC, ADDR

Jumps: label L, goto L
 Conditional jumps: if a goto L

18

Example: SPIM

- Three address code
- We are going to use a subset as a mid-level intermediate code
- Loading/Storing
 - **lw register, addr** - moves value into register
 - **li register, num** - moves constant into register
 - **la register, addr** - moves address of variable into register
 - **sw register, addr** - stores value from register

19

Examples

li \$t2, 5 – load the value 5 into register t2
lw \$t3, x – load value stored at location labeled 'x' into register t3
la \$t3, x – load address of location labeled 'x' into register t3
lw \$t0, (\$t2) – load value stored at address stored in register t2 into register t0
lw \$t1, 8(\$t2) – load value stored at address stored in register 2 + 8 into register t1

20

- Lots of registers – choose which to use for intermediate code generation e.g. 8 registers (\$t0 - \$t7)
- Binary arithmetic operators – work done in registers ($\text{reg1} = \text{reg2 op reg3}$) – reg3 can be a constant
 - **add reg1, reg2, reg3**
 - **sub reg1, reg2, reg3**
 - **mul reg1, reg2, reg3**
 - **div reg1, reg2, reg3**
- Unary arithmetic operators ($\text{reg1} = \text{op reg2}$)
 - **neg reg1, reg2**

21

Intermediate Code Operands

- The operands in 3-address code can be:
 - Program variables (i.e., **v**)
 - Values at memory locations (i.e., **[v]**)
 - Constants or literals (i.e., **42**)
 - Temporary variables (i.e., **t1**)
- Temporary variables = new locations
 - Used to store intermediate values
 - Needed because 3-address code not as expressive as high-level languages

22

Example Code

How could we convert this **if** statement to intermediate code?

if (a < b);		
a = 1;	t1 = a < b	(LT, t1, a, b)
endif;	t2 = !t1	(NOT, t2, t1)
	if (t2) goto L1	(IF, t2, L1)
	a = 1	(ASSIGN, a, 1)
	L1:	(Label, L1)

23

Example Code

How could we convert this *if* statement to intermediate code?

```

if (a < b);
  a = 1;
else;
  a = 2;
endif;

```

t1 = a < b	(LT, t1, a, b)
t2 = !t1	(NOT, t2, t1)
if (t2) goto L1	(IF, t2, L1)
a = 1	(ASSIGN, a, 1)
goto L2	(GOTO, L2)
L1:	(Label, L1)
a = 2	(ASSIGN, a, 2)
L2:	(Label, L2)

24

Example Code

How could we convert this *if* statement to intermediate code?

```

if (a < b+7);
  a = 1;
else;
  a = 2;
endif;

```

t1 = b + 7	(ADD, t1, b, 7)
t2 = a < t1	(LT, t2, a, t1)
t3 = !t2	(NOT, t3, t2)
if (t3) goto L1	(IF, t3, L1)
a = 1	(ASSIGN, a, 1)
goto L2	(GOTO, L2)
L1:	(Label, L1)
a = 2	(ASSIGN, a, 2)
L2:	(Label, L2)

25

Exercise

Convert the following program to an intermediate representation using quadruples.

```

int x = 3;
while (x < 10);
  x = x + 1;
endwhile;

```

26

Exercise

Convert the following program to our intermediate representation using quadruples.

```

int x = 3;
while (x < 10);
  x = x + 1;
endwhile;

```

x = 3	(ASSIGN, x, 3)
L1:	(Label, L1)
t1 = x < 10	(LT, t1, x, 10)
t2 = !t1	(NOT, t2, t1)
if (t2) goto L2	(IF, t2, L2)
t3 = x + 1	(ADD, t3, x, 1)
x = t3	(ASSIGN, x, t3)
goto L1	(GOTO, L1)
L2:	(Label, L2)

27

Translating AST to IC

May have nested language constructs
E.g., while nested within an if statement

Need an algorithmic way to translate
Strategy for each AST node
AST node → sequence of intermediate instructions

Solution

- Start from the AST representation
- Define translation for each node in AST
- Recursively translate nodes

28

Notation

Use the following notation:

[[e]] = the intermediate code representation of construct e

[[e]] may actually be translated into a sequence of intermediate code instructions

If e is an expression, it may represent a value, so we can have: **t = [[e]]**

For variable v: **t = [[v]]** is just the copy instruction, and the same as **t = v**

29

Translating Expressions

Binary operations: $t = [[e_1]] \text{ OP } [[e_2]]$
 - (arithmetic, logical operations and comparisons)

$t_1 = [[e_1]]$
 $t_2 = [[e_2]]$
 $t = t_1 \text{ OP } t_2$

Unary operations: $t = \text{OP } [[e]]$

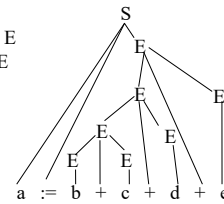
$t_1 = [[e_1]]$
 $t = \text{OP } t_1$

30

Example TAC for Expressions

Generate TAC:

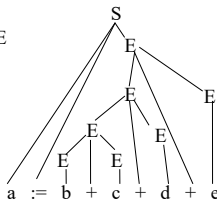
Grammar:
 $S \rightarrow \text{id} := E$
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$



31

Example TAC for Expressions

Grammar:
 $S \rightarrow \text{id} := E$
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$



Generate TAC:
 $t_1 = b$
 $t_2 = c$
 $t_3 = t_1 + t_2$
 $t_4 = d$
 $t_5 = t_3 + t_4$
 $t_6 = e$
 $t_7 = t_5 + t_6$
 $a = t_7$

32

Translating Short-Circuit OR

Short-circuit OR: $t = [[e_1]] \text{ SC-OR } [[e_2]]$
 e.g., `||` operator in C/C++

$t = [[e_1]]$
 if t goto L1
 $t = [[e_2]]$
 L1:

semantics:
 1. evaluate e1
 2. if e1 is true, then done
 3. else evaluate e2

33

Short-circuit and?

Short-circuit AND: $t = [[e_1]] \text{ SC-AND } [[e_2]]$
 e.g., `&&` operator in C/C++

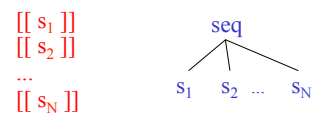
3-address code?

Semantics:
 1. Evaluate e1
 2. if e1 is true,
 then evaluate e2
 3. else done

34

Translating Statements

○ Statement sequence: $[[s_1; s_2; \dots; s_N]]$



○ Intermediate code instructions of a statement sequence is just the concatenation of the intermediate code instructions for each statement

35

Switch Statements

[[switch (e) case $v_1:s_1$, ..., case $v_N:s_N$]]

```

t = [[ e ]]
L1:
c = t != v1
if (c) goto L2
[[ s1 ]]
jump Lend    // if there is a break
L2:
c = t != v2
if (c) goto L3
[[ s2 ]]
jump Lend    // if there is a break
...
Lend:
    
```

36

Issues

- These translations are straightforward
- But, inefficient:
 - Lots of temporaries
 - Lots of labels
 - Lots of instructions
- Can we do this more intelligently?

37

Generating Intermediate Code

- Just as with typechecking, we need to use the syntax of the input to generate the output.
 - Declarations
 - Expressions
 - Control flow
 - Procedure call/return

38

Processing Declarations

- Declarations generate code! = need to allocate memory for variables e.g. A 4-byte word for each initial value, or a large space.
- Global variables vs. local variables
- Binding name to storage location
- Basic types: integer, boolean ...
- Composite types: records, arrays ...
- Tied to expression code generation

39

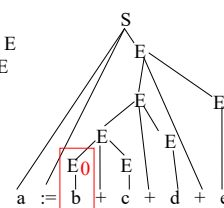
Issues in Processing Expressions

- Generation of correct code
- Type checking/conversions
- Address calculation for constructed types (arrays, records, etc.)
- Expressions in control structures

40

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$



Generate:
 $t0 = b$

As we parse, generate IC for the given input. Use attributes to pass information about temporary variables up the tree

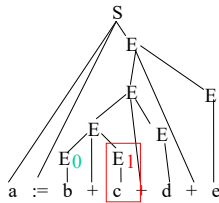
41

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$

Each number
 corresponds to a
 temporary variable.



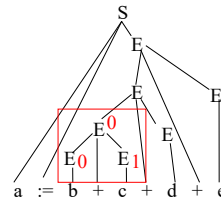
42

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$
 $t0 = t0 + t1$

Each number
 corresponds to a
 temporary variable.



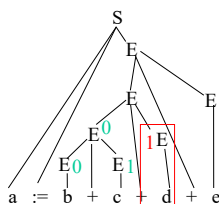
43

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$
 $t0 = t0 + t1$
 $t1 = d$

Each number
 corresponds to a
 temporary variable.



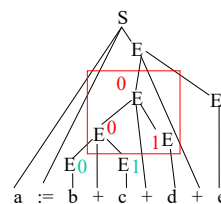
44

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$
 $t0 = t0 + t1$
 $t1 = d$
 $t0 = t0 + t1$

Each number
 corresponds to a
 temporary variable.



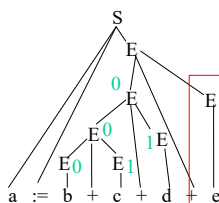
45

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$
 $t0 = t0 + t1$
 $t1 = d$
 $t0 = t0 + t1$
 $t1 = e$

Each number
 corresponds to a
 temporary variable.



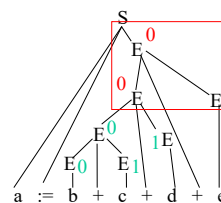
46

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Generate:
 $t0 = b$
 $t1 = c$
 $t0 = t0 + t1$
 $t1 = d$
 $t0 = t0 + t1$
 $t1 = e$
 $t0 = t0 + t1$

Each number
 corresponds to a
 temporary variable.

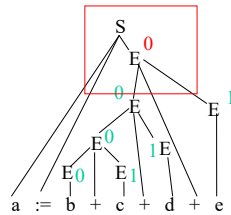


47

Expressions

Grammar:
 $S \rightarrow id := E$
 $E \rightarrow E + E$
 $E \rightarrow id$

Each number corresponds to a temporary variable.



Generate:

```
t0 = b
t1 = c
t0 = t0 + t1
t1 = d
t0 = t0 + t1
t1 = e
t0 = t0 + t1
a = t0
```

48

What about constructed types?

- For basic types, we may be able to just load the value.
- When processing declarations for constructed types, need to keep enough information to generate code that finds the appropriate data at runtime
 - Records
 - Arrays
 - ...

49

Records

- Typical implementation: allocate a block large enough to hold all record fields

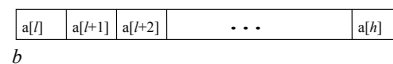

```
struct s{
    type1 field-1;
    ...
    typen field-n;
} data_object;
```
- Boundary issues
- Field names – address will be offset from record address

50

1-D arrays

$a[l..h]$ with element size s

- Number of elements: $e = h - l + 1$
- Size of array: $e * s$
- Address of element $a[i]$, assuming a starts at address b and $l \leq i \leq h$:
 $b + (i - l) * s$



51

Example

$a[3..100]$ with element size 4

- Number of elements: $100 - 3 + 1 = 98$
- Size of array: $98 * 4 = 392$
- Address of element $a[50]$, assuming a starts at address 100

$$100 + (50 - 3) * 4 = 288$$



52

Arrays

- Typical implementation: large block of storage of appropriate size
- Row major vs. column major
- Consider $a[4..6, 3..4]$

Address	Row	Column
$b + 0s$	$a[4,3]$	$a[4,3]$
$b + 1s$	$a[4,4]$	$a[5,3]$
$b + 2s$	$a[5,3]$	$a[6,3]$
$b + 3s$	$a[5,4]$	$a[4,4]$
$b + 4s$	$a[6,3]$	$a[5,4]$
$b + 5s$	$a[6,4]$	$a[6,4]$

53

2-D Arrays: Row Major

- $A[4..7,3..4]$

	Address	Row
$a[4,x]$	$b + 0s$	$a[4,3]$
	$b + 1s$	$a[4,4]$
$a[5,x]$	$b + 2s$	$a[5,3]$
	$b + 3s$	$a[5,4]$
$a[6,x]$	$b + 4s$	$a[6,3]$
	$b + 5s$	$a[6,4]$
$a[7,x]$	$b + 6s$	$a[7,3]$
	$b + 7s$	$a[7,4]$

54

2-D arrays – Row major

$a[l_1..h_1, l_2..h_2]$ with element size s

- Number of elements: $e = e_1 * e_2$, where $e_1 = (h_1 - l_1 + 1)$ and $e_2 = (h_2 - l_2 + 1)$
- Size of array: $e * s$
- Size of each dimension (stride):
 $d_1 = e_2 * s$
 $d_2 = s$
- Address of element $a[i,j]$, assuming a starts at address b and $l_1 \leq i \leq h_1$ and $l_2 \leq j \leq h_2$:
 $b + (i - l_1) * d_1 + (j - l_2) * s$

55

Example

$A[3..100,4..50]$ with elements size 4

- $98 * 47 = 4606$ elements
- $4606 * 4 = 18424$ bytes long
- $d_2 = 4$ and $d_1 = 47 * 4 = 188$
- If a starts at 100, $a[5,5]$ is:
 $100 + (5-3) * 188 + (5-4) * 4 = 480$

56

3-D Arrays

- $a[4..7,3..4,8..9]$
- Size of third (rightmost) dimension = s
- Size of second dimension = $s * 2$
- Size of first dimension = $s * 2 * 2$

$a[4,x]$	$b + 0s$	$a[4,3,8]$	$a[4,3,x]$
	$b + 1s$	$a[4,3,9]$	
	$b + 2s$	$a[4,4,8]$	$a[4,4,x]$
	$b + 3s$	$a[4,4,9]$	
	$b + 4s$	$a[5,3,8]$	$a[5,3,x]$
$a[5,x]$	$b + 5s$	$a[5,3,9]$	
	$b + 6s$	$a[5,4,8]$	$a[5,4,x]$
	$b + 7s$	$a[5,4,9]$	
	$b + 8s$	$a[6,3,8]$	$a[6,3,x]$
$a[6,x]$	$b + 9s$	$a[6,3,9]$	
	$b + 10s$	$a[6,4,8]$	$a[6,4,x]$
	$b + 11s$	$a[6,4,9]$	
	$b + 12s$	$a[7,3,8]$	$a[7,3,x]$
$a[7,x]$	$b + 13s$	$a[7,3,9]$	
	$b + 14s$	$a[7,4,8]$	$a[7,4,x]$
	$b + 15s$	$a[7,4,9]$	

57

3-D arrays – Row major

$a[l_1..h_1, l_2..h_2, l_3..h_3]$ with element size s

- Number of elements: $e = e_1 * e_2 * e_3$, where $e_i = (h_i - l_i + 1)$
- Size of array: $e * s$
- Size of each dimension (stride):
 $d_1 = e_2 * d_2$
 $d_2 = e_3 * d_3$
 $d_3 = s$
- Address of element $a[i,j,k]$, assuming a starts at address b and $l_1 \leq i \leq h_1$ and $l_2 \leq j \leq h_2$ and $l_3 \leq k \leq h_3$:
 $b + (i - l_1) * d_1 + (j - l_2) * d_2 + (k - l_3) * s$

58

Example

$A[3..100,4..50,1..4]$ with elements size 4

- $98 * 47 * 4 = 18424$ elements
- $18424 * 4 = 73696$ bytes long
- $d_3 = 4$, $d_2 = 4 * 4 = 16$ and $d_1 = 16 * 47 = 752$
- If a starts at 100, $a[5,5,2]$ is:
 $100 + (5-3) * 752 + (5-4) * 16 + (2-1) * 4 = 1624$

59

N-D arrays – Row Major

$a[l_1..h_1, \dots, l_n..h_n]$ with element size s

- Number of elements: $e = \prod e_i$ where $e_i = (h_i - l_i + 1)$
- Size of array: $e * s$
- Size of each dimension (stride):
 $d_i = e_{i+1} * d_{i+1}$
 $d_n = s$
- Address of element $a[i_1, \dots, i_n]$, assuming a starts at address b and $l_j \leq i_j \leq h_j$:
 $b + (i_1 - l_1) * d_1 + \dots + (i_n - l_n) * d_n$

60

Flow of Control In Three Address Code

61

Understanding the while loop

while Test do Statement

standard translation:

```
* evaluate test
if test is true
  execute statement
  go to *
else
  go to **
** ...
```

... but no "else" statement
in three-address code

62

Understanding the while loop

while Test do Statement

standard translation:

```
* evaluate test
if test is true
  execute statement
  go to *
else
  go to **
** ...
```

alternative translation:

```
* evaluate test
if test is false
  go to **
execute statement
go to *
** ...
```

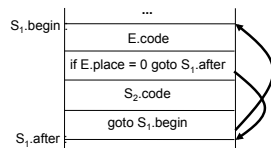
... but no "else" statement
in three-address code

63

Generating the code for while

$S_1 \rightarrow \text{while } E \text{ do } S_2$

```
* evaluate test
if test is false
  go to **
execute statement
go to *
** ...
```



```
S1.begin := newlabel();
S1.after := newlabel();
S1.code := gen(S1.begin, ".") || E.code ||
  gen("if" E.place " = 0 goto" S1.after) ||
  S2.code ||
  gen("goto" S1.begin) ||
  gen(S1.after, ".")
```

NOTE:
 $E.place$ = name that holds value of E
 $E.code$ = sequence of 3-address statements
 $newlabel()$ = returns a unique new temp variable
 gen = accepts a string and produces it as a 3-address quadruple
 $||$ = concatenation operator that joins address code segments.

64

if statement

$S_1 \rightarrow \text{if } T \text{ then } S_2$

```
evaluate test
if test is false
  go to **
execute statement
** ...
```

```
S1.next := newlabel();
S1.code := T.code ||
  gen("if" T.place " = 0 goto" S1.after) ||
  S2.code ||
  gen(S1.after, ".")
```

65

if-else statement

- $S_1 \rightarrow \text{if } T \text{ then } S_2 \text{ else } S_3$

```
evaluate test
if test is false
  go to *
execute s2
go to **
* execute s3
** ...
```

```
S1.else := newlabel();
S1.next := newlabel();
S1.code := T.code ||
  gen("if" T.place "= 0 goto" S1.else) ||
  S2.code ||
  gen("goto" S1.next) ||
  gen(S1.else ".")
S3.code ||
gen(S1.next ".")
```

66

for statement

- $S_1 \rightarrow \text{for } R \text{ do } S_2$
- $R \rightarrow \text{id} = \text{int}_1 \text{ to } \text{int}_2$

```
initialise limit
initialise counter
* if counter > limit
  go to **
execute s2
increment counter
go to *
** ...
```

```
S1.begin := newlabel();
S1.next := newlabel();
S1.limit := newtemp();
S1.code := gen(S1.limit "=" R.end) ||
  gen(R.var "=" R.start) ||
  gen(S1.begin ".") ||
  gen("if" R.var ">" S1.limit "goto" S1.next) ||
  S2.code ||
  gen(R.var "=" R.var "+1") ||
  gen("goto" S1.begin) ||
  gen(S1.next ".")
```

```
R.start := int1.value
R.end := int2.value
R.var := id.place
```

67

SPIM examples:

68

Spim Addressing Modes

Format	Address =
(register)	contents of register
imm	immediate
imm(register)	immediate + contents of register
symbol	address of symbol
symbol +/- imm	address of symbol + or - immediate
symbol +/- imm(register)	address of symbol + or - (immediate + contents of register)

We typically only use some of these in our intermediate code

69

$a := b * -c + b * -c$

lw \$t0, b

b
t0

70

$a := b * -c + b * -c$

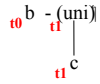
lw \$t0, b
lw \$t1, c

b
t0
c
t1

71

$a := b * -c + b * -c$

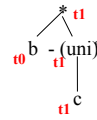
```
lw $t0,b
lw $t1,c
neg $t1,$t1
```



72

$a := b * -c + b * -c$

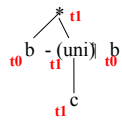
```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
```



73

$a := b * -c + b * -c$

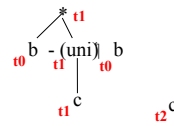
```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
```



74

$a := b * -c + b * -c$

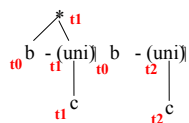
```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
lw $t2,c
```



75

$a := b * -c + b * -c$

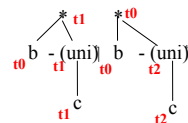
```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
lw $t2,c
neg $t2,$t2
```



76

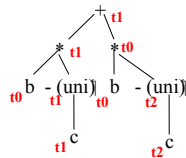
$a := b * -c + b * -c$

```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
lw $t2,c
neg $t2,$t0
mul $t0,$t0,$t2
```



77

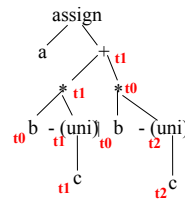
$a := b * -c + b * -c$



```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
lw $t2,c
neg $t2,$t0
mul $t0,$t0,$t2
add $t1,$t0,$t1
```

78

$a := b * -c + b * -c$

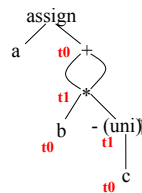


```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1, $t1,$t0
lw $t0,b
lw $t2,c
neg $t2,$t0
mul $t0,$t0,$t2
add $t1,$t0,$t1
sw $t1,a
```

79

$a := b * -c + b * -c$

```
lw $t0,b
lw $t1,c
neg $t1,$t1
mul $t1,$t1,$t0
add $t0,$t1,$t1
sw $t0,a
```



80

• Comparison operators

set condition – temp1 = temp2 xxx temp3, where xxx is a condition (gt, ge, lt, le, eq) – temp1 is 0 for false, non-zero for true.

– **sgt reg1,reg2,reg3**

– **slt reg1,reg2,reg3**

– ...

81

More Spim

- Jumps
 - **b label** – unconditional branch to label
 - **bxxx temp, label** – conditional branch to label, **xxx** = condition such as eqz, neq, ...
- Procedure statement
 - **j al label** – jump and save return address
 - **jr register** – jump to address stored in register

82

Control Flow

```
while x <= 100 do      lw $t0,x
    x := x + 1          li $t1,100
end while              L25: sle $t2,$t0,$t1
                      beqz $t2,L26
                      addi $t0,$t0,1
                      sw $t0,x
                      b L25
L26:                   loop body
```

branch if false

83

Example: Generating Prime Numbers

```
print 2 print blank
for i = 3 to 100
  divides = 0
  for j = 2 to i/2
    if j divides i evenly then divides = 1
  end for
  if divides = 0 then print i print blank
end for
exit
```

84

Loops

```
print 2 print blank
for i = 3 to 100
  divides = 0
  for j = 2 to i/2
    if j divides i evenly then divides = 1
  end for
  if divides = 0 then print i print blank
end for
exit
```

85

Outer Loop: **for i = 3 to 100**

```
li $t0, 3      # variable i in t0
li $t1, 100    # max loop counter in t1
11: sle $t7, $t0, $t1  # i <= 100
    beqz $t7, 12
    ...
    addi $t0, $t0, 1 # increment i
    b 11
12:
```

86

Inner Loop: **for j = 2 to i/2**

```
li $t2, 2      # j = 2 in t2
div $t3, $t0, 2 # i/2 in t3
13: sle $t7, $t2, $t3  # j <= i/2
    beqz $t7, 14
    ...
    addi $t2, $t2, 1  # increment j
    b 13
14:
```

87

Conditional Statements

```
print 2 print blank
for i = 3 to 100
  divides = 0
  for j = 2 to i/2
    if j divides i evenly then divides = 1
  end for
  if divides = 0 then print i print blank
end for
exit
```

88

if j divides i evenly then divides = 1

```
rem $t7, $t0, $t2  # remainder of i/j
bnez $t7, 15       # if there is
                  # remainder
li $t4, 1          # divides=1 in t4
15:
...
bnez $t4, 16       # if divides = 0 not prime
print i
16:
```

89

SPIM System Calls

- Write(i)


```
li $v0,1
lw $a0,i
syscall
```
- Read(i)


```
li $v0,5
syscall
sw $v0,i
```
- Exiting


```
li $v0,10
syscall
```

90

Example: Generating Prime Numbers

```
print 2 print blank
for i = 3 to 100
  divides = 0
  for j = 2 to i/2
    if j divides i evenly then divides = 1
  end for
  if divides = 0 then print i print blank
end for
exit
```

91

```
.data
blank: .asciiz " "

.text
li $v0,1
li $a0,2
syscall          # print 2
li $v0,4
la $a0,blank     # print blank
syscall

li $v0,1
lw $a0,i
syscall          # print i

li $v0,10
syscall          # exit
```

92

```
.data
blank: .asciiz " "

.text
main:
li $v0,1
li $a0,2
syscall
li $v0,4
la $a0,blank
syscall

li $t0,3          # i in t0
li $t1,100        # max in t1
sle $t7,$t0,$t1
beqz $t7,$t2
li $t2,0

li $t2,2          # j in t2
div $t3,$t0,$t2   # max in t3
sle $t7,$t2,$t3
beqz $t7,$t4
rem $t7,$t0,$t2
bnez $t7,$t5
li $t4,1

addi $t2,$t2,1
b t3              #end of inner loop

14:
bnez $t4,$t6
addi $t0,$t0,1
b t1              #end of outer loop

12:
li $v0,10
syscall

16:
addi $t0,$t0,1
b t1
```

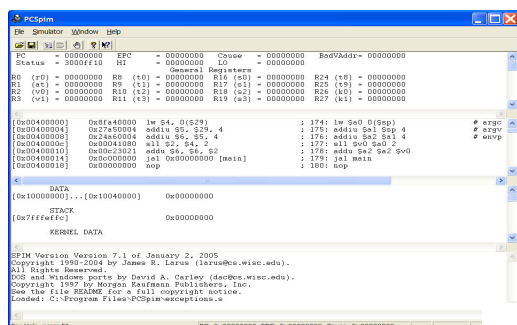
Entire program

inner loop

93

PC SPIM

can run by providing an input file



94

Processing Control Structures

- Constructs:
 - If
 - While
 - Repeat
 - For
 - case
- Label generation – all labels must be unique
- Nested control structures – need a stack

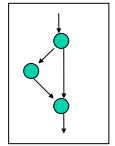
95

Conditional Examples

```

if (y > 0) then begin
    lw $t0,y
    li $t1,0
    sgt $t2,$t0,$t1    # = 1 if true
    beqz $t2,L2
    ...body...
L2:
end

```



Control Flow

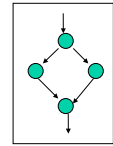
96

Conditional Examples

```

if (y > 0) then begin
    lw $t0,y
    li $t1,0
    sgt $t2,$t0,$t1    # = 1 if true
    beqz $t2,L2
    ...body-1...
    b L3
L2:
    ...body-2 ...
L3:
end else
    ...body-2 ...
end

```



Control Flow

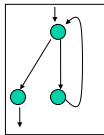
97

Looping constructs

```

while x < 100 do
    L25: lw $t0,x
        li $t1,100
        sge $t2,$t0,$t1
        beqz $t2,L26
        ... body ...
        b L25
    L26:
end

```



Control Flow

98

Generating Conditionals

```

if_stmt → IF expr THEN
    { code to eval expr ($2) already done
      get two new label names
      output conditional ($2=false) branch to first
      label }
stmts ELSE
    { output unconditional branch to second
      label
      output first label }
stmts ENDIF
    { output second label }

```

99

Generating Loops

```

for_stmt → FOR id = start TO stop
    { code to eval start ($4) and stop ($6) done
      get two new label names
      output code to initialize id = start
      output label1
      output code to compare id to stop
      output conditional branch to label2 }
stmts END
    { increment id (and save)
      unconditional branch to label1
      output label2 }

```

100

Nested conditionals

- Need a stack to keep track of correct labels
- Can implement own stack
 - push two new labels at start of statement
 - pop two labels when end statement
 - while generating code, use the two labels on the top of the stack
- Can use YACC
 - Give two tokens (like IF and THEN) label types.
 - At start of statement, when generate new labels, assign them to these tokens
 - When you need the numbers for generation, just use the value associated with the token.

101